

Agent Substitution v0.1 — Short-list + Routing

Recommendation

- You are reading: **the v0.1 results report** — the short-list, per-task routing, and blended savings after applying the bar
- Upstream: [v0 results report](#) — the quality × cost table (results after running)
- Source table: [per-task matrix CSV](#) — 18×4 (score + token + \$), the source of every number here
- Evidence: [run-log](#) · bug write-up: [findings](#)
- Bar: Paul locked **quality $\geq 70\%$ / cost $\leq \frac{1}{2}$** (a starting value, see [DECISIONS](#))
- Scope: the common-14 task set, consistent with and comparable to the v0 main table
- Numbers: all recomputed line-by-line against the source table, fully traceable

0. Bottom line

Under the bar Paul locked (quality $\geq 70\%$ / cost $\leq \frac{1}{2}$), the cheap-model short-list is GLM-4.6 only.

But the real product conclusion isn't "swap to GLM or not" — it's **how to route**:

- Swap everything to GLM (no routing) = **76% quality / 8.4× cheaper**;
- Route per task (fall back to Claude on hard tasks) = **99% quality / ~2× cheaper**.

The latter loses almost no quality and still halves the cost, and it lands right on the $\leq \frac{1}{2}$ line Paul set — this is the sweet spot hybrid routing is meant to deliver, and it's how v0.1 moves the project from "can it be replaced" to "how to replace it most cost-effectively."

1. Short-list: applying the bar (common-14)

Model	Quality	= Claude	\$/task	= Claude cost	Cheaper	Pass?
Claude sonnet-4.6	0.653	100%	\$0.439	1.00	1×	baseline
GLM-4.6	0.497	76%	\$0.052	0.12	8.4×	✅ short-list
DeepSeek V3.2	0.433	66%	\$0.162	0.37	2.7×	❌ fails quality
Qwen3-Coder	0.377	58%	\$0.209	0.48	2.1×	❌ fails quality

Quality is the only filter. DeepSeek and Qwen already meet the $\leq \frac{1}{2}$ cost bar — what stops them is the 70% quality line. So under this bar the short-list is clean (GLM only), and our external framing is straightforward: **filtered purely on quality, not gated by a cost threshold.**

2. Per-task tiers: 10 of 14 tasks go to GLM, 4 stay on Claude

We compared GLM against Claude task by task, with the bar set to "GLM's score on this task $\geq$ 70% of Claude's." The result splits cleanly into two groups:

- **10 GLM-safe tasks.** GLM meets or exceeds the bar on these, so they can go straight to the cheap model — on financial_stmt, pe_screening, os_log, and tcga GLM mostly scored 100% or higher, and k8s, homework, agora and others also clear the line.
- **4 Claude-only tasks.** GLM clearly cannot do these, so Claude must be kept: causal_ihdp_ite_estimation (Claude 0.178 / GLM 0.000), crf_sdtm_mapping (0.589 / 0.000), molecular_structure_plausibility (0.500 / 0.167), capacitated_vehicle_routing (1.000 / 0.000).

This shows the right form of routing is not "swap Claude out for GLM wholesale" but "route per task": send what GLM can handle to GLM, fall back to Claude on the hard tasks. This 10-vs-4 split is the first prototype of the routing policy table.

3. Blended savings curve ★ (the core conclusion)

Policy	Quality	= Claude	\$/task	Cheaper
All-Claude (baseline)	0.653	100%	\$0.439	1×
Pure GLM (no routing)	0.497	76%	\$0.052	8.4×
Per-task hybrid / verify fallback	0.647	99%	~\$0.22	~2×

In one line: swapping to GLM trades away 24% of quality for 8.4× cheaper; **routing per task gives up only 1% of quality and still saves ~2×** — hard tasks fall back to Claude, so almost no quality is lost while cost is halved. And this hybrid point lands right on the $\leq \frac{1}{2}$ cost line Paul set (0.5× cost / 99% quality), which means his starting bar is an **achievable sweet spot** under per-task routing, not an arbitrary number.

Routing mechanism: the cheap model runs first → auto-scored → falls back to Claude if it doesn't pass (verify-then-route). Quality matches oracle routing (0.647); cost is slightly higher (\$0.22, because fallback tasks are paid for twice).

4. Limitations

- "99% / 2×" is an **achievable upper bound, not a production measurement**. It was obtained on the 14 ALE tasks using the real scorer to simulate routing; in production there is no ready-made scorer, so the actual result depends on the accuracy of the classifier and the verify proxy — it will land between "pure GLM at 76%" and "the 99% upper bound."
- **Small sample:** common-14; the trend is reliable, but absolute values await a larger N.

- **Edge case:** on epidemiology_forecast both models score 0, so it is counted as GLM-safe (both fail → route to the cheap model with no quality loss) — handled as expected and noted.
 - **Distribution:** still the public ALE benchmark, ≠ BFC's real traffic; the final answer needs real usage data to follow.
-

5. Next steps (who does what)

SDE (Abby) — turn the source table into a router. Every file you need to run or open is given by path below (relative to the repo root); the scripts use only the Python standard library (no dependencies to install) and are all run from the repo root. Start by opening `analysis/README.md`.

Step 1 · reproduce and confirm alignment (can start right away, no dependence on the bar)

- From the repo root, run: `python3 analysis/apply_threshold.py`
- It reads: `reports/v0-per-task-matrix.csv` (the 18×4 source table, committed)
- It produces: prints the §1 short-list, §2 tiers, and §3 savings curve; and writes `reports/routing-policy.csv` (per task → route to GLM or Claude)
- What you should get (alignment check): short-list GLM 76% / 8.4x, with DeepSeek and Qwen both failing on quality; tiers 10 GLM-safe / 4 Claude-only; curve pure-GLM 76% / 8.4x, per-task hybrid 99% / ~2x. Matching this report = aligned.
- To try a different bar: edit the `Q_BAR` / `C_BAR` constants at the top of `analysis/apply_threshold.py` and re-run.
- To understand where the numbers come from: see `analysis/README.md` (which script maps to which section, the prices used) and `analysis/extract_matrix.py` (the matrix's provenance; note it reads raw `.logs/` that are local-only, not in the repo).
- **Done = your numbers match the report, and `reports/routing-policy.csv` is generated.**

Step 2 · verify-then-route prototype (safety-net layer, no dependence on the bar, in two parts)

You must first pin down "auto-score with what," or you'll get stuck:

- (a) **Offline (do this first; pure data, no live model runs):** simulate "GLM runs first, falls back to Claude if it doesn't pass" on these 14 tasks, reproducing the 99% / 2x line. The logic is already in `analysis/apply_threshold.py` — specifically the `blended()` function and the policy named `Verify-then-route (realistic)`; adapt that. **Done = a prototype that reproduces 99% / 2x.**
- (b) **Production:** there's no ALE scorer in production, so design an accept/reject proxy (a judge model or a heuristic) to stand in for it and decide "should this GLM output be accepted." Validate it against the `route_to` column in `reports/routing-policy.csv`. **Done = the proxy design + its accuracy on the 14 tasks at deciding "fall back or not."**

Step 3 · lightweight classifier (design first, not a trained model)

- Classify the user prompt into task type plus difficulty. The label space is ALE's 8 domains, readable directly from the `task` column prefix in `reports/v0-per-task-matrix.csv`: `business_finance` / `computing_math` / `health_medicine` / `life_sciences` / `legal` / `physical_sciences` / `education_info` / `transport_safety`.

- Attach a confidence score, and default to keeping the task on Claude when confidence is low, so a misclassification doesn't cause a misroute.
- Training data is thin (only these 14 tasks, one or two per domain), so this phase is about **locking the design, label space, and confidence threshold** — not training a model. Done = a one-pager design + label space + low-confidence fallback rule.

Step 4 · real-traffic phase to follow

- Check our computed costs against the models' official pricing (current prices are in `analysis/README.md` 's price table), infer task type from tokens or channel, and wire in online success/failure signals and instrumentation — moving from "offline tiering on a public benchmark" to "online refinement on our own traffic."

First two moves: open `analysis/README.md` and read it, then run `python3 analysis/apply_threshold.py` from the repo root.

Elena (PM): owns delivery of this report and the applied table (short-list plus routing recommendation).

Paul: $\leq 1/2$ is a starting value. Once the SDE has the real routing results from verify-then-route, or once we're on real traffic data, decide whether the line needs adjusting.

Not in scope this phase: no expanding N, no adding Windows/GPU tasks, no re-testing Kimi (see [DECISIONS](#)).

Data and full evidence trail: repo `agent-eval-v0` ([source CSV](#) / [run-log](#) / [findings](#)). The bar decision is in [DECISIONS](#).

Repo: <https://github.com/yingshill/agent-eval-v0>